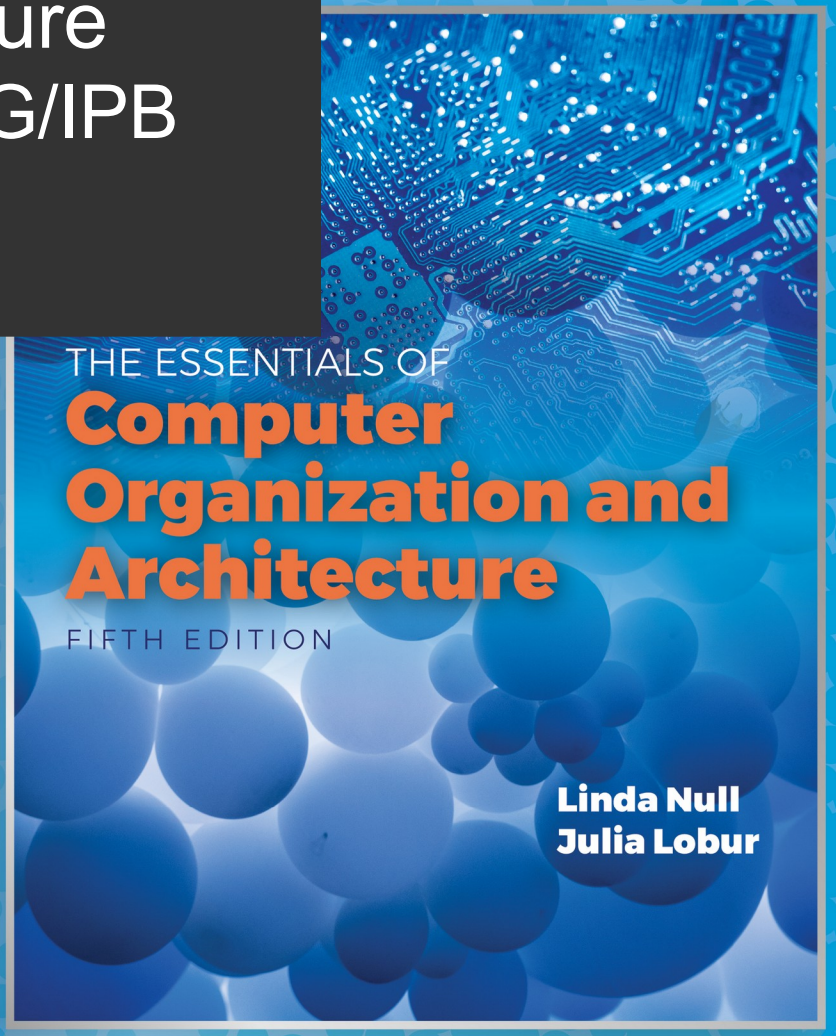
Computer Architecture
Informatics Eng., ESTiG/IPB
2025/2026

José Rufino

(mainly based on Chapter 2 of the book "The Essentials of Computer Organization and Architecture" of Linda Null & Julia Lobur)



Unit 2: **Data Representation**

Objectives

- Understand the fundamentals of numerical data **representation** and **manipulation** in digital computers
- Master the **conversion** between various **radix systems**
- Understand how errors can occur in computations because of **overflow** and **truncation**
- Understand the fundamental concepts of **floating-point representation**
- Know the main **character codes**

2.1 Introduction

- ***bit***: the most basic unit of information in a computer
  - bit = **bi**nary dig**it**
  - a state of "on" or "off" in a digital circuit
  - … or of "high" or "low" voltage level

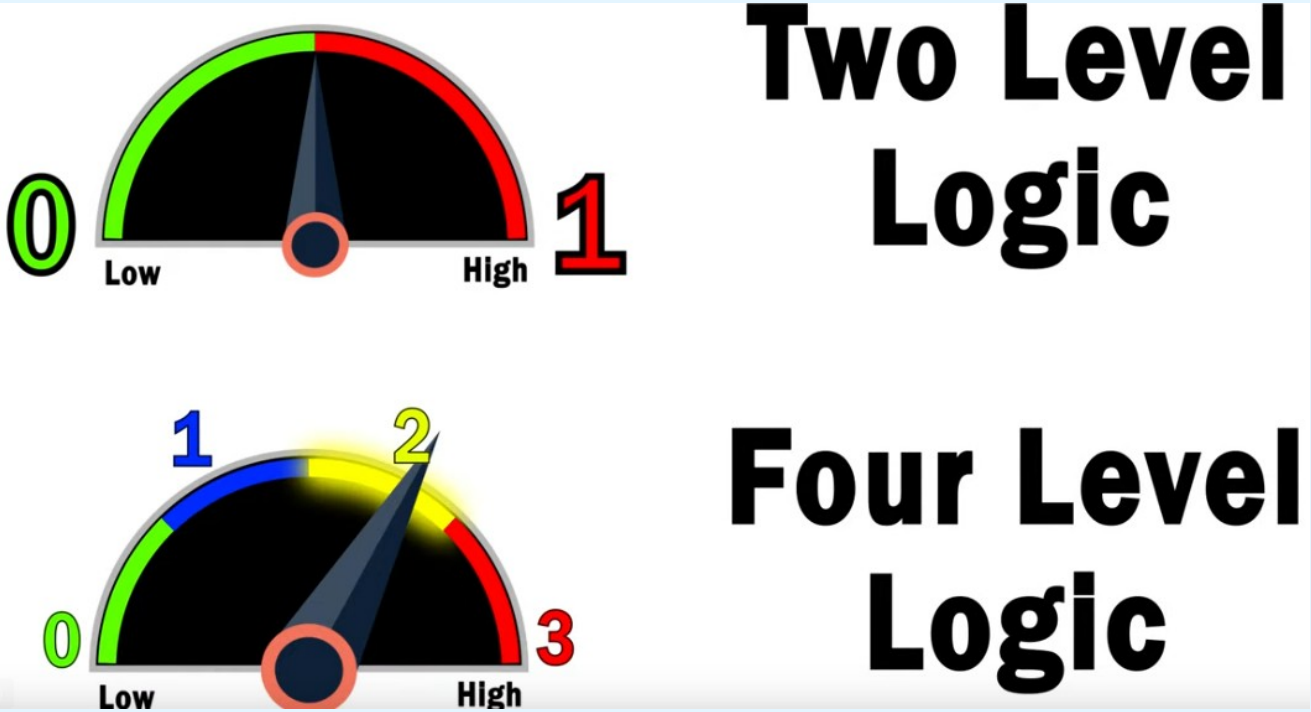
- ***byte***: is a group of eight bits (IBM System/360, 1964)
  - the smallest possible *addressable* unit of computer storage
  - the word "addressable" means that a particular byte can be accessed (read/write) by providing its location in memory

2.1 Introduction

- ***word***: a contiguous group of bytes
  - *words* can be any number of bits or bytes
  - word sizes of 16, 32, or 64 bits are most common
  - in a "word-addressable system", a word is the smallest addressable unit of storage
- ***nibble***: a contiguous group of four bits
  - therefore, bytes consist of two nibbles: a "high-order" (most significant) nibble, and a "low-order" (least significant) nibble
  - example: the byte 10100101 has the nibbles 1010 and 0101

2.1 Introduction

- Why do computers use the binary system ?



Why can't computers use base 3 instead of binary? Voltage states explained.
https://www.youtube.com/watch?v=fXwSFhUVFmE

2.2 Positional Numbering Systems

- A **number** is a sequence of digits/numerals, expressed in a certain numerical **base (radix)**

- The base defines the **alphabet** (admissible digits)
  - base 10: 10 digits (0,1,2,...,9) - decimal system
  - base 2: 2 digits (0,1) - binary system
  - base 3: 3 digits (0,1,2) - ternary system
  - base 8: 8 digits (0,1,...,7) - octal system
  - base 16: 16 digits (0,1,...,9,A,B,C,D,E,F) - hexadecimal system

2.2 Positional Numbering Systems

- Positional Numbering Systems are also known as **Weighted** Numbering Systems:
  - each digit, in a certain position in the number, is multiplied by a certain power of the number radix
    - each digit is a coefficient of a certain power
  - the value of the power depends on the digit's position
- Using the base in subscript allows to distinguish numbers expressed in different bases ($10_{10} \neq 10_2$)
  - base 10 is assumed implicitly if subscript is empty

2.2 Positional Numbering Systems

- 5836.47_{10} expressed as a sum of products (of coefficients {0,1,…,9} multiplied by powers of 10):

$$5 \times 10^3 + 8 \times 10^2 + 3 \times 10^1 + 6 \times 10^0 + 4 \times 10^{-1} + 7 \times 10^{-2}$$

- 11001.11_2 expressed as a sum of products (of coefficients {0,1} multiplied by powers of 2):

$$1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} =$$

$$16 + 8 + 0 + 0 + 1 + 0.5 + 0.25 = 25.75_{10}$$

2.3 Conversions Between Different Bases

- Note: 2.3 applies only to **non-negative integers**

- Two methods for radix conversion:
  - the **subtraction method**
    - more intuitive ... but requires knowledge of the radix powers of the target radix
    - akin to express the converted number as a sum of powers of the target radix
  - the **division remainder** method (see slide 18)

2.3 Conversions Between Different Bases - The Subtraction Method

- **Converting 155_{10} to base 2**
  - express 155 as a sum of powers of 2, in which the weight (coefficient) of each power can only be 0 or 1 (the symbols of binary alphabet)
  - as $2^8 = 256 > 155$, the result will have less than 8 digits
  - for the power $2^7 = 128$, we have $\mathbf{1} \times 128 = 128 \leq 155$
  - so, we use the **1** and remove 128 from 155, and the remainder is 27

$$\begin{array}{r} 155 \\ -\ 128 \\ \hline 27 \end{array} = 2^7 \times \mathbf{1}$$

2.3 Conversions Between Different Bases - The Subtraction Method

- **Converting 155_{10} to base 2**
  - the next power of 2 ($2^6 = 64$) is > 27, so its coefficient will be **0** and still remains 27
  - the next power of 2 ($2^5 = 32$) is > 27, so its coefficient will be **0** and still remains 27
  - the next power of 2 ($2^4 = 16$) is <= 27, so its coefficient will be **1** and still remains 11

155
- 128 = 2^7 x **1**
27

- 0 = 2^6 x **0**
27

- 0 = 2^5 x **0**
27

- 16 = 2^4 x **1**
11

2.3 Conversions Between Different Bases - The Subtraction Method

- **Converting 155_{10} to base 2**
  - the next power of 2 ($2^3 = 8$) is <= 11, so its coefficient will be **1** and still remains 3
  - the next power of 2 ($2^2 = 4$) is > 3, so its coefficient will be **0** and still remains 3
  - the next power of 2 ($2^1 = 2$) is <= 3, so its coefficient will be **1** and still remains 1
  - the next power of 2 ($2^0 = 1$) is <= 1, so its coefficient will be **1** and remains 0

$$\begin{array}{r} 155 \\ -\ 128 \\ \hline 27 \end{array} = 2^7 \times 1$$

...

$$\begin{array}{r} 11 \\ -\ 8 \\ \hline 3 \end{array} = 2^3 \times 1$$

$$\begin{array}{r} 3 \\ -\ 0 \\ \hline 3 \end{array} = 2^2 \times 0$$

$$\begin{array}{r} 3 \\ -\ 2 \\ \hline 1 \end{array} = 2^1 \times 1$$

$$\begin{array}{r} 1 \\ -\ 1 \\ \hline 0 \end{array} = 2^0 \times 1$$

2.3 Conversions Between Different Bases - The Subtraction Method

- **Converting 155_{10} to base 2**
  - the result of the conversion, read **top to bottom**, is

    $155_{10} = \mathbf{10011011_2}$
  - basically, this method translates on expressing the number to convert as a sum of powers of 2:

    $155 = 2^7 + 2^4 + 2^3 + 2^1 + 2^0$
  - the bits in the positions given by the exponents will be 1, and the others will be 0

155
- 128 = 2^7 x **1**
27
- 0 = 2^6 x **0**
27
- 0 = 2^5 x **0**
27
- 16 = 2^4 x **1**
11
- 8 = 2^3 x **1**
3
- 0 = 2^2 x **0**
3
- 2 = 2^1 x **1**
1
- 1 = 2^0 x **1**
0

2.3 Conversions Between Different Bases - The Subtraction Method

- **Converting 190_{10} to base 3**
  - express 190 as a sum of powers of 3, in which the weight (coefficient) of each power can be 0, 1 or 2 (the symbols of ternary alphabet)
  - as $3^5 = 243 > 190$, the result will have less than 6 digits
  - for the power $3^4 = 81$, we have **2** x 81 = 162 ≤ 190
  - so, we use the **2** and remove 162 from 190, with a remainder of 28

$$\begin{array}{r} 190 \\ -\ 162 \\ \hline 28 \end{array} = 3^4 \times 2$$

2.3 Conversions Between Different Bases - The Subtraction Method

- **Converting 190_{10} to base 3**
  - the next power of 3 still unused is $3^3 = 27$
  - the coefficient 2 cannot be used because 2 x 27 > 28
  - but the coefficient **1** can be used once **1** x 27 ≤ 28 ; so we use the **1** and subtract 27 from 28, with the remainder being 1
  - the next power of 3 ($3^3 = 9$) is too big, so its coefficient will be **0**; it still remains 1

$$
\begin{array}{rl}
190 & \\
-\ 162 & = 3^4 \times 2 \\
\hline
28 & \\
-\ 27 & = 3^3 \times 1 \\
\hline
1 & \\
-\ 0 & = 3^2 \times 0 \\
\hline
1 &
\end{array}
$$

2.3 Conversions Between Different Bases - The Subtraction Method

- **Converting 190_{10} to base 3**
  - the next power of 3 ($3^1 = 3$) is again too large, so the coefficient will be **0** and still remains 1
  - for the last power of 3 ($3^0 = 1$), the coefficient **1** allows **1** x $3^0 = 1 \leq 1$; we use the **1** and now the remainder is 0
  - the conversion result, read **top to bottom**, is:

    $190_{10} = \mathbf{21001}_3$

190		
− 162	$= 3^4 \times$	2
28		
− 27	$= 3^3 \times$	1
1		
− 0	$= 3^2 \times$	0
1		
− 0	$= 3^1 \times$	0
1		
− 1	$= 3^0 \times$	1
0		

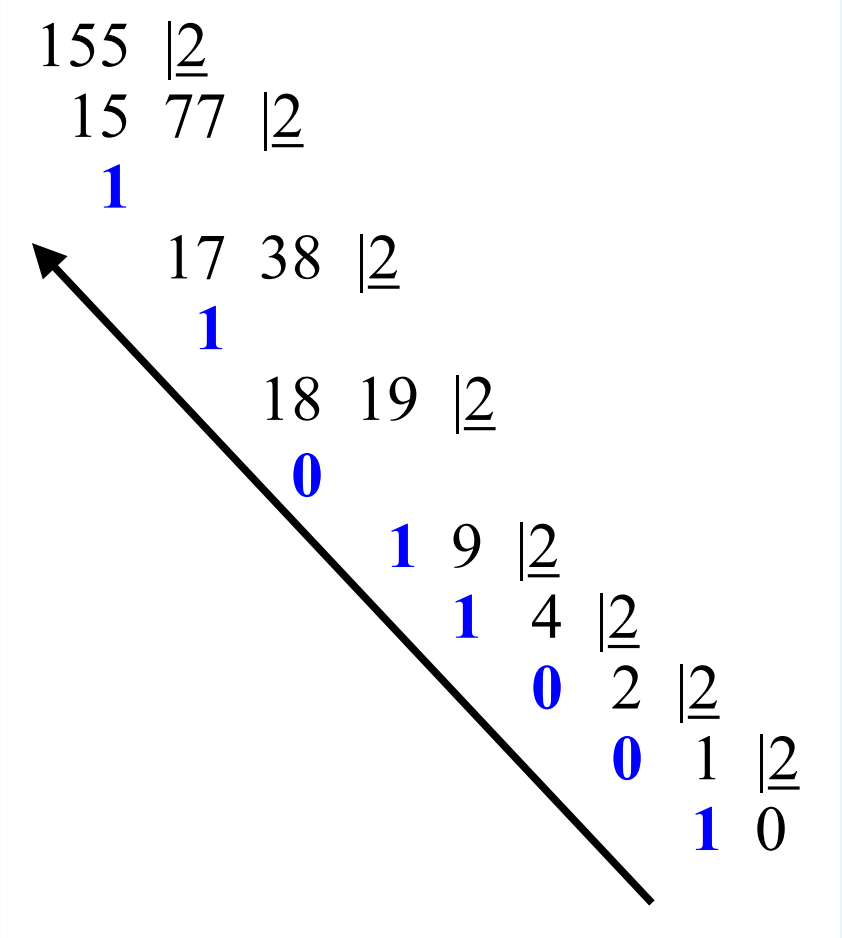
2.3 Conversions Between Different Bases - The Division Remainder Method

- The **division remainder** method is easy to automate; also, no need to know the powers of the target radix
- Rationale:
  - successive division by a base is equivalent to successive subtraction by powers of the base
  - the remainders of the successive divisions are the digits of the converted number in the target radix
- Lets use the division remainder method to repeat the same conversions (155 to base 2, 190 to base 3)

2.3 Conversions Between Different Bases - The Division Remainder Method

- **Converting 155_{10} to base 2**
  - take 155 and divide it by 2
  - the quotient (77) is > 0, so we repeat the process, until the quotient becomes 0
  - the result of the conversion are the remainders, read **bottom to top** :

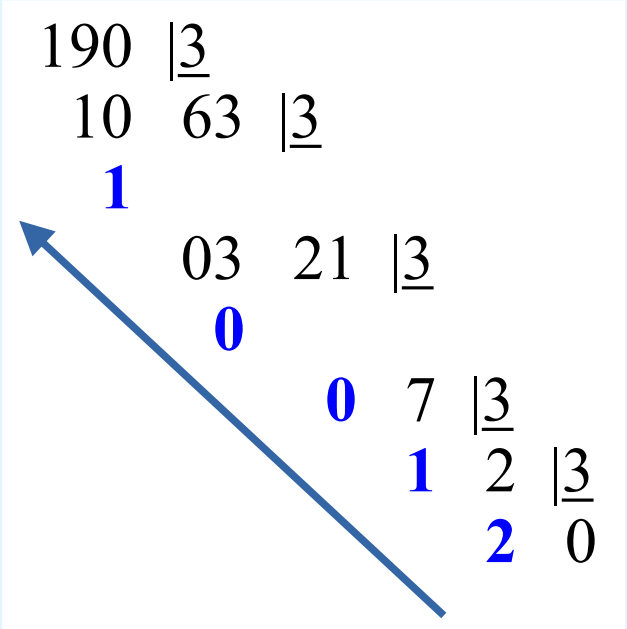
$$155_{10} = \mathbf{10011011_2}$$



2.3 Conversions Between Different Bases - The Division Remainder Method

- **Converting 190_{10} to base 3**
  - take 190 and divide it by 3
  - the quotient (63) is > 0, so we repeat the process, until the quotient becomes 0
  - the result of the conversion are the remainders, read **bottom to top** :

$$190_{10} = \mathbf{21001_3}$$



2.3 Conversions Between Different Bases - Conversion of Fractional Values

- Fractional values have an integer part and a fractional part (for instance: 5836.47)
- Fractional values can be approximated in all bases
- Unlike integer values, fractions do not necessarily have exact representations under all radixes
  - **example**: the quantity ½ is exactly representable in bases 2 and 10, but not in base 3

2.3 Conversions Between Different Bases - Conversion of Fractional Values

- Fractional decimal values have nonzero digits to the right of the **decimal point**
- Fractional values of other radix systems have nonzero digits to the right of the ***radix point***
- Numerals to the right of a *radix point* are coefficients of negative powers of the radix:

$$0.47_{10} = 4 \times 10^{-1} + 7 \times 10^{-2}$$

$$0.11_2 = 1 \times 2^{-1} + 1 \times 2^{-2}$$

$$= \frac{1}{2} + \frac{1}{4}$$

$$= 0.5 + 0.25 = 0.75$$

2.3 Conversions Between Different Bases - Conversion of Fractional Values

- As with whole-number conversions, there are two conversion methods: **subtraction** and **multiplication**

- The **subtraction method**:
  - identical to the subtraction method for whole numbers ...
  - … but instead of subtracting positive powers of the target radix, we subtract negative powers of the radix
    - start with the largest value first, n^{-1}, where n is the target radix, and continue using larger negative exponents

2.3 Conversions Between Different Bases - Conversion of Fractional Values - Subtraction Method

- **Converting 0.8125_{10} to binary**

  - the result, read from **top to bottom**, is:

$$0.8125_{10} = 0.1101_2$$

$$\begin{array}{rl} 0.8125 & \\ -\ 0.5000 & = 2^{-1} \times 1 \\ \hline 0.3125 & \\ -\ 0.2500 & = 2^{-2} \times 1 \\ \hline 0.0625 & \\ -\ 0 & = 2^{-3} \times 0 \\ \hline 0.0625 & \\ -\ 0.0625 & = 2^{-4} \times 1 \\ \hline 0 & \end{array}$$

2.3 Conversions Between Different Bases - Conversion of Fractional Values - Subtraction Method

- **This method works with any target base, not just binary**

- **Converting 0.4304_{10} to base 5**
  - the result, read from **top to bottom**, is:

$$0.4304_{10} = 0.\mathbf{2034}_{5}$$

$$\begin{aligned}
&0.4304 \\
-\ &\underline{0.4000} = 5^{-1} \times \mathbf{2} \\
&0.0304 \\
-\ &\underline{0.0000} = 5^{-2} \times \mathbf{0} \\
&0.0304 \\
-\ &\underline{0.0240} = 5^{-3} \times \mathbf{3} \\
&0.0064 \\
-\ &\underline{0.0064} = 5^{-4} \times \mathbf{4} \\
&0.0000
\end{aligned}$$

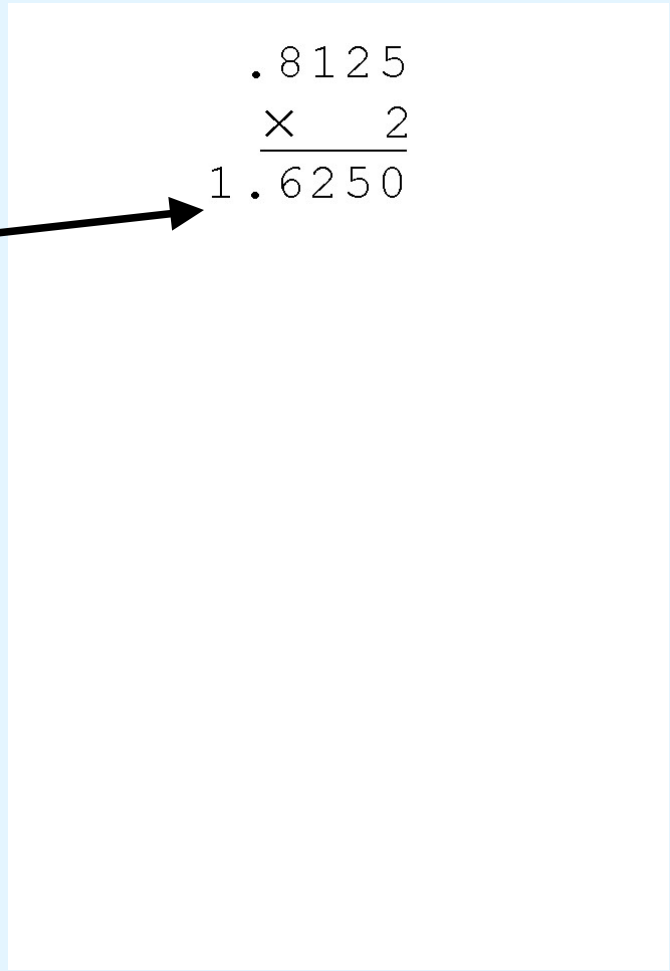
2.3 Conversions Between Different Bases - Conversion of Fractional Values

- The **multiplication method**:
  - multiply the original fractional value by the target radix
  - in the outcome, the value to the left of the radix point (integer part of the product) is the 1st digit of the number
  - the value to the right of the radix point will now be multiplied by the target radix
  - in the outcome, the value to the left of the radix point is the 2nd digit of the number
  - ...
  - the process repeats until the value to be multiplied becomes zero, or the required precision is reached

2.3 Conversions Between Different Bases - Conversion of Fractional Values - Multiplication Method

- **Converting 0.8125_{10} to binary**

  - the first product carries 1 into the units place



2.3 Conversions Between Different Bases - Conversion of Fractional Values - Multiplication Method

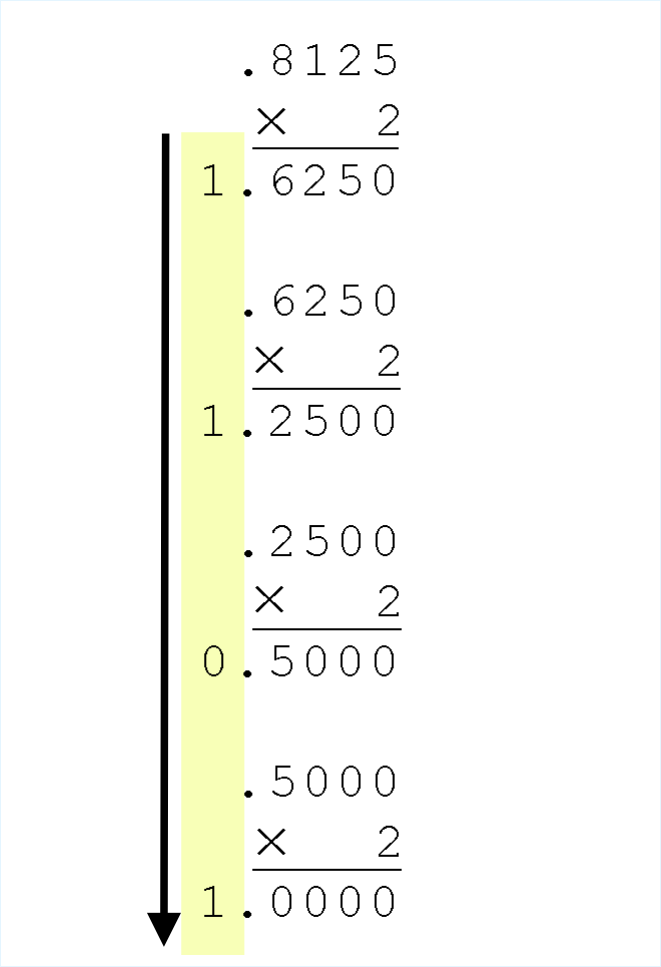
- **Converting 0.8125_{10} to binary**
  - ignoring the value in the units place at each step, continue multiplying each fractional part by the radix

```
  .8125
   ×   2
1.6250

  .6250
   ×   2
1.2500

  .2500
   ×   2
0.5000
```

2.3 Conversions Between Different Bases - Conversion of Fractional Values - Multiplication Method

- **Converting 0.8125_{10} to binary**
  - the process finishes when the product is zero, or if the desired number of binary places was reached
  - the result, read from **top to bottom**, is:

$$0.8125_{10} = 0.\mathbf{1101}_2$$



2.3 Conversions Between Different Bases - Conversion of Fractional Values - Multiplication Method

- **This method works with any target base, not just binary**
- **Converting 0.4304_{10} to base 5**
  - the result, read from **top to bottom**, is:

$$0.4304_{10} = 0.\mathbf{2034}_{5}$$

.4304
x 5
2.1520
.1520
x 5
0.7600
.7600
x 5
3.8000
.8000
x 5
4.0000

2.3 Conversions Between Different Bases - Auxiliary Table

n	2^n	3^n	4^n	5^n	6^n	7^n	8^n	9^n
-5	0,03125	0,0041152263	0,0009765625	0,00032	0,0001286008	0,000059499	3,05176E-005	1,69351E-005
-4	0,0625	0,012345679	0,00390625	0,0016	0,0007716049	0,0004164931	0,0002441406	0,0001524158
-3	0,125	0,037037037	0,015625	0,008	0,0046296296	0,0029154519	0,001953125	0,0013717421
-2	0,25	0,1111111111	0,0625	0,04	0,0277777778	0,0204081633	0,015625	0,012345679
-1	0,5	0,3333333333	0,25	0,2	0,1666666667	0,1428571429	0,125	0,1111111111
0	1	1	1	1	1	1	1	1
1	2	3	4	5	6	7	8	9
2	4	9	16	25	36	49	64	81
3	8	27	64	125	216	343	512	729
4	16	81	256	625	1296	2401	4096	6561
5	32	243	1024	3125	7776	16807	32768	59049
6	64	729	4096	15625	46656	117649	262144	531441
7	128	2187	16384	78125	279936	823543	2097152	4782969
8	256	6561	65536	390625	1679616	5764801	16777216	43046721
9	512	19683	262144	1953125	10077696	40353607	134217728	387420489
10	1024	59049	1048576	9765625	60466176	282475249	1073741824	3486784401

checkpoint:

exercises 2.1 to 2.4

2.3 Conversions Between Different Bases - Quick Conversions between Bases 2, 8 and 16

- The **binary** numbering system is the most important radix system for digital computers
  - binary numbers are the basis for all data representation in digital computers ...
  - … and so knowing the binary numbering system is fundamental to understand the operation of all computer components as well as to the design of instruction set architectures

2.3 Conversions Between Different Bases - Quick Conversions between Bases 2, 8 and 16

- However, it is difficult to read long strings of binary numbers, and even a modestly-sized decimal number becomes a very long binary number
  - for example: $11010100011011_2 = 13595_{10}$

- For compactness and ease of reading, binary values are usually expressed in hexadecimal (base-16), or octal (or base-8) representation

2.3 Conversions Between Different Bases - Quick Conversions between Bases 2, 8 and 16

- To convert from **binary to hexadecimal**, all we need to do is group the binary digits into **groups of four**

- Example: the hexadecimal representation of the binary number 11010100011011_2 $(= 13595_{10})$ is

0011	0101	0001	1011
3	5	1	B

2.3 Conversions Between Different Bases - Quick Conversions between Bases 2, 8 and 16

- To convert from **binary to octal**, all we need to do is group the binary digits into **groups of three**

- Example: the octal representation of the binary number 11010100011011_2 $(= 13595_{10})$ is

011	010	100	011	011
3	2	4	3	3

2.3.1 Addendum
- Representation Range in Base 2 (non-negatives)

- For binary numbers of N bits, without signal (non-negative), the representation range is:

$$\{ 0, \ldots, 2^N - 1 \}$$

- Examples:
  - for N=8 bits

$$\{ 0, \ldots, 2^8 - 1 \} = \{ 0, \ldots, 255 \}$$

  - for N=7 bits

$$\{ 0, \ldots, 2^7 - 1 \} = \{ 0, \ldots, 127 \}$$

2.3.1 Addendum - Binary Arithmetic

- Binary arithmetic is similar to decimal arithmetic (note: we'll only revisit examples with 2 numbers)
- Decimal addition
  - if the sum in a column is >=10, there is a carry
  - carry => add ***1*** unit to the next column

Carry	**0**	**0**	**1**	**0**	**1**
	5	6	7	1	9
+	3	1	8	6	3
Sum	**8**	**8**	**5**	**8**	**2**

- Binary addition
  - bit rules →

0 + 0 = **0**

0 + 1 = **1**

1 + 1 = **0** **with a carry of 1**

<=> 1 + 1 = 10

  - example →

Carry	**1**	**1**	**0**	**1**
	1	1	0	1
+	1	1	0	1
Sum	**1**	**1**	**0**	**1**	**0**

2.3.1 Addendum
- Binary Arithmetic

- Decimal subtraction
  - in a column if the subtrahend > minuend, there will be a borrow
  - borrow => add ***1*** to the subtrahend of the next column

Borrow		*1*	*1*	*1*	
Minuend		8	3	0	3
Subtrahend	-	5	4	8	6
Difference		**2**	**8**	**1**	**7**

- Binary subtraction
  - bit rules →

```
0 - 0 = 0
1 - 1 = 0
1 - 0 = 1
0 - 1 = 1 borrows 1
```

  - example →

Borrow		*1*	*1*		
Minuend		1	1	0	1	1
Subtrahend	-		1	1	0	1
Difference		**0**	**1**	**1**	**1**	**0**

2.4 Signed Integer Representation

- The conversions we have so far presented have involved only non-negative numbers
- To represent negative values, computers use the **high-order bit** to indicate the **sign** of a value
  - high-order bit: the leftmost bit (most significant bit) in a byte
- The **remaining bits** contain the **value** of the number:

S V V V V V . . . V

- Ways in which signed binary numbers may be expressed:
  - Signed magnitude
  - One's complement
  - Two's complement

[https://www.youtube.com/watch?v=4qH4unVtJkE]

2.4 Signed Integer Representation - Signed Magnitude

- Bits to the right of the sign bit: they hold the binary representation of the absolute value of the number
- Range of representation with N = 1 + (N-1) bits

$$\{ -(2^{N-1}-1), \ldots, -0, +0, \ldots, (2^{N-1}-1) \}$$

- Example: in a word with N=8 bits, under signed magnitude, the absolute value is in the N-1=7 less significant bits (those to the right of the sign bit)
  - +3 will be represented as 00000011
  - - 3 will be represented as 10000011
  - range of representation:

$$\{ -127, \ldots, -0, +0, \ldots, 127 \}$$

2.4 Signed Integer Representation - One's Complement

- Representation of integers in 1's complement:
  - positive number: none conversion needed; the value of the number is directly read from its representation
  - negative number: represented as the 1's complement (all bits flipped) of that number modulus; thus:
    - to discover the modulus of a negative number, we just need to calculate it's 1's complement (flipping its bits)
- example (with number of 8 bits):
  - **+**3 is represented in 1's complement as: **0**0000011
  - **-** 3 is represented in 1's complement as: **1**1111100
  - **-** 3 is represented in signed magnitude as: **1**0000011

[https://www.compscilib.com/calculate/ones-complement]

2.4 Signed Integer Representation - Two's Complement

- Representation of integers in 2's complement:
  - number > 0: none conversion needed; the value of the number is directly read from its representation
  - number < 0: find the one's complement of the number and then add 1

- Example (with number of 8 bits):
  - **+**3 is represented in 2's complement as: **0**0000011
  - **-** 3 is represented in 1's complement as: **1**1111100
  - **-** 3 is represented in 2's complement as: **1**1111101

[https://www.compscilib.com/calculate/twos-complement]

2.4 Signed Integer Representation - Representation Ranges

- Representation range with N = 1 + (N-1) bits
  - signed magnitude and 1's complement

$$\{ -(2^{N-1}-1), \ldots, -0, +0, \ldots, (2^{N-1}-1) \}$$

  - 2's complement

$$\{ -2^{N-1}, \ldots, 0, \ldots, (2^{N-1}-1) \}$$

    - example: N=4 bits : { -8, ..., 0, ..., 7}
- Representation of Zero:
  - Signed magnitude: 0 0...0 (+0) ; 1 0...0 (-0)
  - 1's complement: 0 0...0 (+0) ; 1 1...1 (-0)
  - 2's complement: 0 0...0

checkpoint:

exercises 2.5, 2.6, 2.7 //,2.16

2.5 Floating-Point Representation

- The signed magnitude, one's complement, and two's complement representation that we have just presented deal with integer values only
- Without modification, these formats are not useful in scientific or business applications that deal with real number values
- Floating-point representation solves this problem

2.5 Floating-Point Representation

- If we are clever programmers, we can perform floating-point calculations using any integer format
- This is called *floating-point emulation*, because floating point values aren't stored as such, we just create programs that make it seem as if floating-point values are being used
- Most of today's computers are equipped with specialized hardware that performs floating-point arithmetic with no special programming required

2.5 Floating-Point Representation

- Floating-point numbers allow an arbitrary number of decimal places to the right of the decimal point
  - For example: 0.5 x 0.0125 = 0.00625

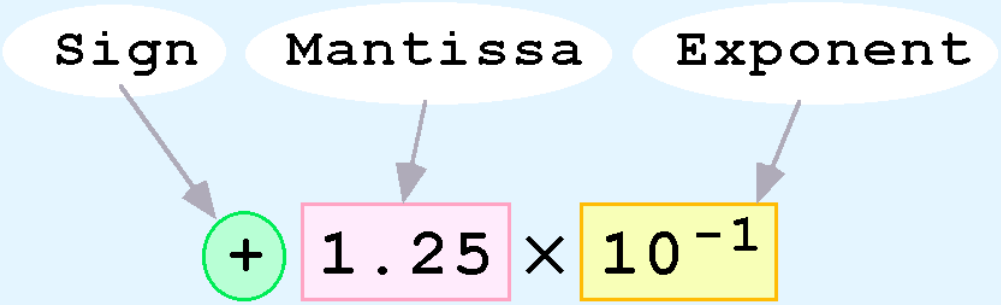
- They are often expressed in **scientific notation**
  - For example:

    $0.0000000125 = 1.25 \times 10^{-8}$

    $5000000000000 = 5.0 \times 10^{12}$

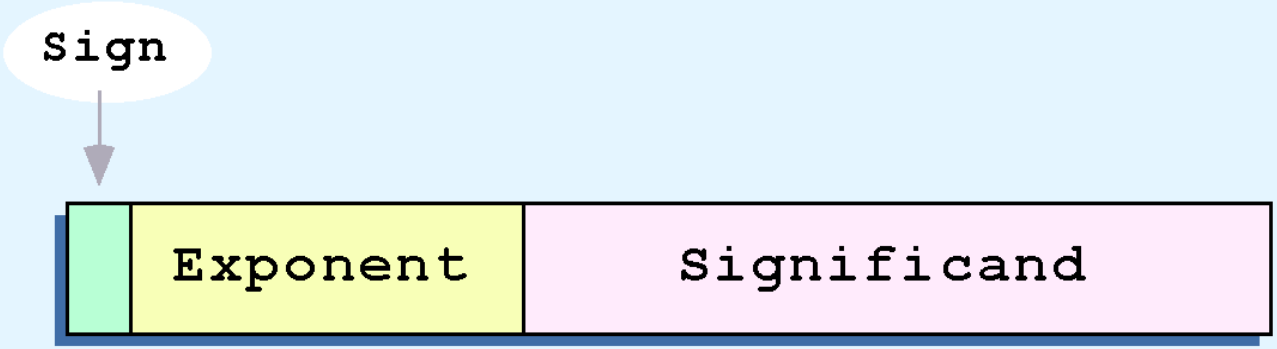
2.5 Floating-Point Representation

- Computers use a form of scientific notation for floating-point representation
- Numbers written in scientific notation have three components:



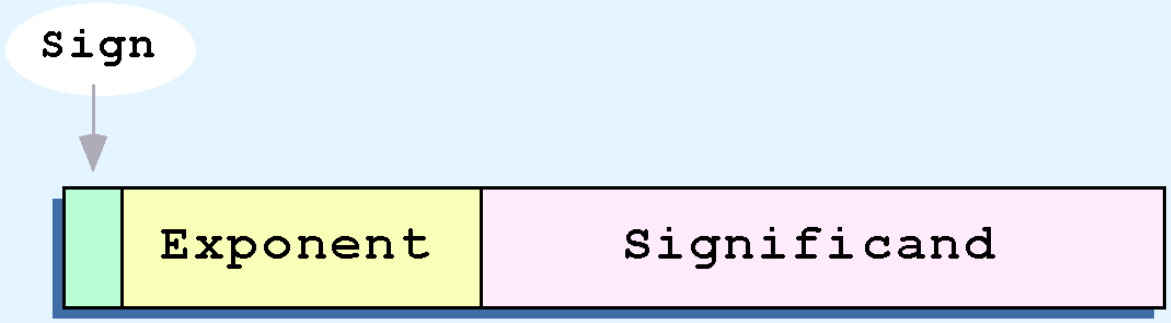
2.5 Floating-Point Representation

- Computer representation of a floating-point number consists of three fixed-size fields:



- This is the standard arrangement of these fields

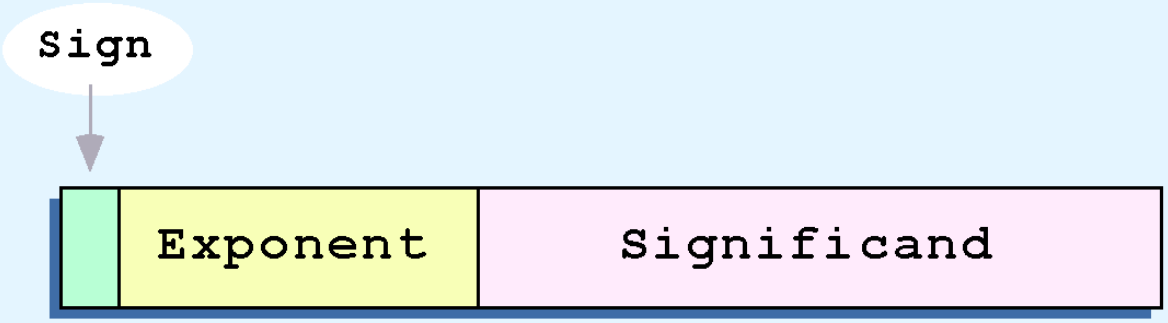
2.5 Floating-Point Representation



- The one-bit **sign** field is the sign of the stored value
- The size of the **exponent** field, determines the range of values that can be represented
- The size of the **significand** determines the precision of the representation

For illustrative purposes, we will use a simple 14-bit model with a 5-bit exponent and an 8-bit significand.

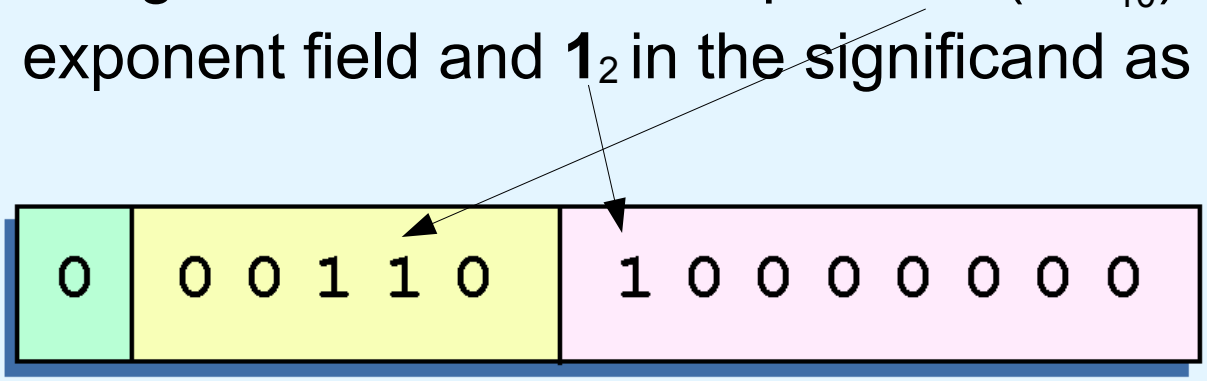
2.5 Floating-Point Representation - 14-bit Model



- The significand of a floating-point number is always preceded by an implied binary point
  - the significand is always a **fractional** (< 1) value
  - to achieve that, move the radix point as necessary, until the significand becomes fractional
- The exponent indicates the power of 2 to which the significand is raised.

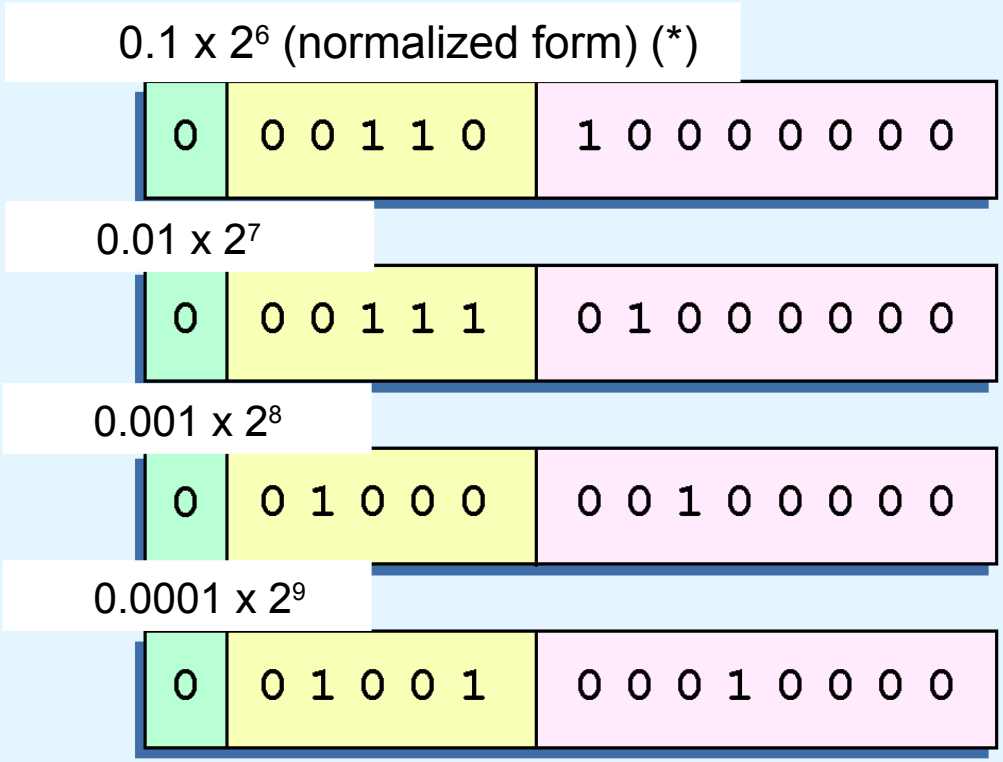
2.5 Floating-Point Representation - 14-bit Model

- **Example0**: representation of $32_{10} = 100000_2$
  - in **(binary) scientific notation** we have $32_{10} = 100000.0_2 \times 2^0 = 10000.00_2 \times 2^1 = \ldots = 10.00000_2 \times 2^4 = 1.000000_2 \times 2^5 = 0.1000000_2 \times 2^6 = 0.\mathbf{1}_2 \times 2^{\mathbf{6}}$
  - using this information, we put $\mathbf{110}_2$ (= $\mathbf{6}_{10}$) in the exponent field and $\mathbf{1}_2$ in the significand as shown



2.5 Floating-Point Representation - 14-bit Model: Normalization

- **Problem:** there are many possible representations for the same number
  - waste of space, and may be ambiguous
- **Example:** all illustrations to the right are possible representations of 32 →



- **Solution**: **normalization** - the 1º bit of the significand must be 1, which results in a unique bit pattern for each value
  - thus, for the example above, we have $32_{10} = 0.1_2 x 2^6$ (*)

2.5 Floating-Point Representation - 14-bit Model: Biased Exponent

- Another **problem** with our system is that we have made no allowances for negative exponents
  - e.g., we have no way to express 0.5 ($=2^{-1}$) ...
  - … once there is no sign bit in the exponent field!

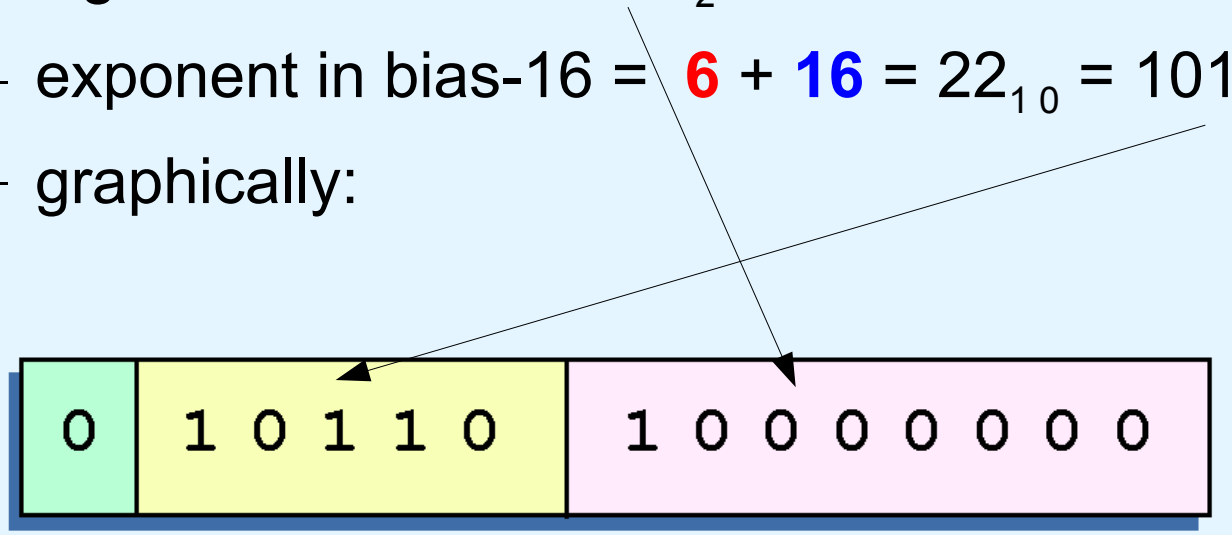
- So, a **possible solution** would be to use a signed integer representation (signed magnitude, one's/ two's complement) for the exponent field; but there's another more efficient approach (see next slide)

2.5 Floating-Point Representation - 14-bit Model: Biased Exponent

- A **better solution** is to adopt a **biased exponent**
  - a **bias** is a number that is approximately midway in the range of values that would be used to express the exponents if all of them were non-negative
  - thus, with a biased exponent we add the bias to the original exponent before storing it in the exponent field, and subtract the bias to recover is true value
  - with a 5-bit exponent we have 32 values (0..31) and use **16** for bias (excess-16 or bias-16 representation)
  - biased exponents 0...15 are originally negative, thus pertaining to fractional numbers
  - this revised version the 14-bits model is henceforth named as **14-bit model with bias-16**

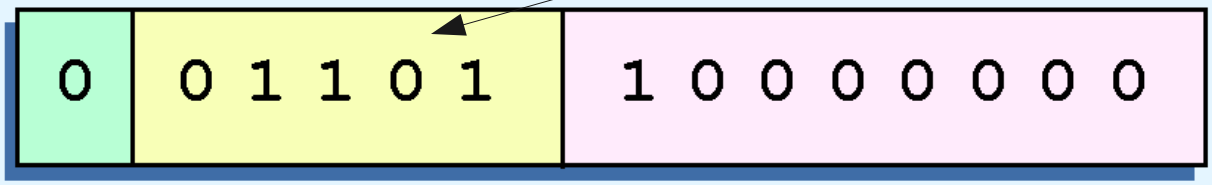
2.5 Floating-Point Representation - 14-bit Model with Bias-16

- **Example1**: representation of 32_{10}
  - normalized binary scientific notation: $32_{10}=0.\mathbf{1}_2 x 2^{6}$
  - significand = $\mathbf{1}0000000_2$
  - exponent in bias-16 = **6** + **16** = 22_{10} = 10110_2
  - graphically:



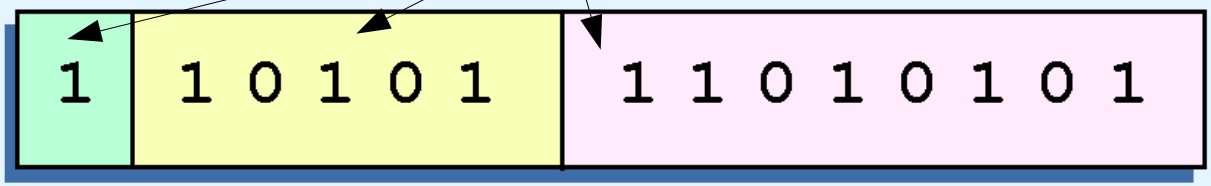
2.5 Floating-Point Representation - 14-bit Model with Bias-16

- **Example2**: representation of 0.0625_{10}
  - we know that 0.0625_{10} is 2^{-4}
  - normalized binary scientific notation:
    $0.0625_{10} = 0.0001_2 \times 2^0 = \ldots = 0.01_2 \times 2^{-2} = 0.\mathbf{1}_2 \times 2^{-3}$
  - significand = $\mathbf{1}0000000_2$
  - exponent in bias-16 = **-3** + **16** = $13_{10} = 01101_2$
- graphically:



2.5 Floating-Point Representation - 14-bit Model with Bias-16

- **Example3**: representation of -26.625_{10}
  - first, we find that $26.625_{10} = 11010.101_2$
  - normalizing, we have: $26.625_{10} = 0.\mathbf{1}1010101 \times 2^{5}$
  - significand = $\mathbf{1}1010101_2$
  - exponent in bias-16 = **5** + **16** = $21_{10} = 10101_2$
  - once $-26.625_{10} < 0$, the sign bit will be **1**
  - graphically:



2.5 Floating-Point Representation - 14-bit Model with Bias-16: Zeros, Ranges

- **zeros**:
  - |0|00000|00000000| = +0
  - |1|00000|00000000| = -0

checkpoint:
exercise 2.18

- **range of positives:**
  - < positive: |0|00000|10000000| = + 0.1x2^(0-**16**=-16) =
    + 0.0000000000000001x2^0 = (*) 0.0000076293945312 5
  - > positive: |0|11111|11111111| = + 0.11111111x2^(31-**16**=15)
    = + 111111110000000.0x2^0 = (*) 32640
- **range of negatives:**
  - < negative: |1|11111|11111111| = - 32640
  - > negative: |1|00000|10000000| = - 0.0000076293945312 5

note: calculation of (*) done in https://www.exploringbinary.com/binary-converter/

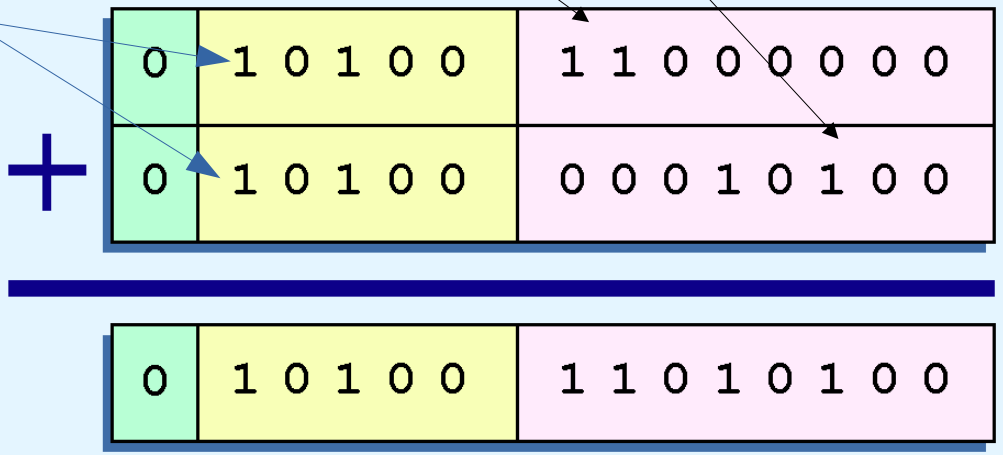
2.5 Floating-Point Representation - Floating-Point Arithmetic

- Floating-point addition and subtraction are done using methods analogous to how humans perform calculations
- First both operands are expressed with the same power
  - If necessary, one of the operands is denormalized (the one with smallest absolute value)
- Significands are added and the exponent is preserved
- If the sum of the significands produces a carry bit (**1**) in the most significant bit column, the significand looses its least significant bit and the carry bit becomes the most significant; moreover, the exponent is added one unit

note: later, a simpler method will be introduced ...

2.5 Floating-Point Representation - 14-bit Model with Bias-16: Addition

- **Example** (without carry): add 12_{10} and 1.25_{10}
  - $12_{10} = 0.1100_2 \times 2^4$
  - $1.25_{10} = 0.101_2 \times 2^1 = 0.000101 \times 2^4$
  - exponent = 4 + 16
    = 20 = 10100_2



  - the sum is
    $0.110101_2 \times 2^4$

2.5 Floating-Point Representation - 14-bit Model with Bias-16: Addition

- **Example** (with carry): add 0.5_{10} to itself
  - $0.5_{10} = 0.\mathbf{1}_2 \times 2^0$
  - exponent = 0 + 16 = 16 = 10000_2
  - sum:

```
                    1
      | 0 | 10000 | 10000000 |
    + | 0 | 10000 | 10000000 |
    ---------------------------
      | 0 | 10000 | 00000000 |
```

  - adjustment: +1 | 10000000 | 0

    | 0 | 10001 | 10000000 | =

    $0.1_2 \times 2^{17-16} = 0.1_2 \times 2^1 = 1.0_2 \times 2^0 = 1_{10}$

checkpoint: exercises 2.20, 2*.1 to 2*.7

2.5 Floating-Point Representation - 14-bit Model with Bias-16: Addition

- **alternative method**: add the denormalized numbers (i.e., with exponent 0) and normalize the result
  - **example** (12 + 1.25 = 13.25):
    - $12_{10} = 1100.0_2 \times 2^0$
    - $1.25_{10} = 1.01_2 \times 2^0$
    - sum:

```
  1100.00
+   1.01
--------
1101.01 = 1101.01x2^0 = 0.110101x2^4
```

    - FP representation: |0|10100|11010100| = 13.25

note: this method is not adequate to computers, once they always store numbers in normalized format and only if really needed (see previous method) will denormalization happen (and restricted to a single number).

$20 = 16 + 4$

2.5 Floating-Point Representation - 14-bit Model with Bias-16: Addition

- **alternative method** (cont.)
  - **example** (0.5 + 0.5 = 1.0):
    - $0.5_{10} = 0.1_2 \times 2^0$
    - sum:

$$\begin{array}{r} 1 \\ 0.1 \\ +0.1 \\ \hline 1.0 = 1.0\times2^0 = 0.1\times2^1 \end{array}$$

    - FP representation: |0|10001|1<u>0000000</u>| = 1.0

17 = 16 + 1

2.5 Floating-Point Representation - 14-bit Model with Bias-16: Addition

- **alternative method** (cont.)
  - **example** (3.125 + 0.6015625 = 3.7265625):
    - $3.125_{10} = 11.001_2 \times 2^0$
    - $0.6015625_{10} = 0.1001101_2 \times 2^0$
    - sum:

```
  11.001
+ 0.1001101
-----------
11.1011101x2^0 = 0.111011101x2^2
```

    - FP representation: |0|10010|11101110|1 = 3,71875

18 = 16 + 2

1/128 lost

2.5 Floating-Point Representation - IEEE-754 Standard

- **simple precision** (32 bits): 1+31 bits
  - 8 bits exponent, with bias-127 (2^7-1)
  - 23 bits significand
- **double precision** (64 bits): 1+63 bits
  - 11 bits exponent, with bias-1023 (2^{10}-1)
  - 52 bits significand
- in the normalized form, **the first bit 1 is implicit**
  - i.e., that bit 1 is not stored in the 1st bit of the significand
  - allows **an extra bit (the rightmost)** for extra precision
  - example of reconstruction of a number (in simple precision):

    | s | eeeeeeee | mmmmmmmmmmmmmmmmmmmmmmm | =
    s 1.mmmmmmmmmmmmmmmmmmmmmmm$_2$x2^(eeeeeeee$_2$-127)

2.5 Floating-Point Representation - IEEE-754 Standard

- examples of simple precision with exact representation
  - | 0 | 01111111 | 00000000000000000000000 | =
    + 1.00000000000000000000000$_2$x2^(01111111$_2$-127) =
    + 1.0$_2$x2^(127$_{10}$-127$_{10}$) = + 1.0$_2$x2^0 = **1.0$_{10}$**
  - | 0 | 01111110 | 00000000000000000000000 | =
    + 1.00000000000000000000000$_2$x2^(01111110$_2$-127$_{10}$)=
    + 1.0$_2$x2^(126$_{10}$-127$_{10}$) = + 1.0$_2$x2^(-1) = + .10$_2$x2^0 = **0.5$_{10}$**
  - | 0 | 10000011 | 00111000000000000000000 | =
    + 1.00111000000000000000000$_2$x2^(10000011$_2$-127$_{10}$)=
    + 1.00111$_2$x2^(131$_{10}$-127$_{10}$) = + 1.00111$_2$x2^4 = + 10011.1$_2$x2^0 = **19.5$_{10}$**
  - | 1 | 10000000 | 11100000000000000000000 | =
    - 1.11100000000000000000000$_2$x2^(10000000$_2$-127$_{10}$)=
    - 1.111$_2$x2^(128$_{10}$-127$_{10}$) = - 1.111$_2$x2^1 = - 11.11$_2$x2^0 = **-3.75$_{10}$**

2.5 Floating-Point Representation - IEEE-754 Standard

- example of simple precision without exact representation
  - | **0** | **01111011** | **10011001100110011001101** | =
    + 1.**10011001100110011001101**$_2$x2^(**01111011**$_2$-127) =
    + 1.**10011001100110011001101**$_2$x2^(**123**$_{10}$-127$_{10}$=-4) =
    + 0.000**1100110011001100110011001101**$_2$x2^0 (*) =

    0.100000001490116119384765625$_{10}$ = closest representation of **0.1**
  - to confirm this value:

    error = 0.100000001490116119384765625
    - compute the sum of products corresponding to (*) - see next slide
    - use https://www.exploringbinary.com/floating-point-converter/
    - print 0.1 as `float` in a C program:

```
#include <stdio.h>
int main() {
  float x=0.1;
  printf("%.30f\n", x); // precision of 30 digits with right zeros
  printf("%.30g\n", x); // precision of 30 digits without right zeros
}
```

```
0.100000001490116119384765625000
0.100000001490116119384765625
```

2.5 Floating-Point Representation - IEEE-754 Standard

- example of simple precision without exact representation
  - conversion of $0.000\mathbf{1}\mathbf{10011001100110011001101}_2$ to base 10

A	B	C	D
exponent	**power of 2**	**bit**	**(power of 2) x (bit)**
−1	0,5	0	0
−2	0,25	0	0
−3	0,125	0	0
−4	0,0625	1	0,0625
−5	0,03125	1	0,03125
−6	0,015625	0	0
−7	0,0078125	0	0
−8	0,00390625	1	0,00390625
−9	0,001953125	1	0,001953125
−10	0,0009765625	0	0
−11	0,00048828125	0	0
−12	0,000244140625	1	0,000244140625
−13	0,0001220703125	1	0,0001220703125
−14	6,103515625E-05	0	0
−15	3,0517578125E-05	0	0
−16	1,52587890625E-05	1	1,52587890625E-05
−17	7,62939453125E-06	1	7,62939453125E-06
−18	3,814697265625E-06	0	0
−19	1,9073486328125E-06	0	0
−20	9,5367431640625E-07	1	9,5367431640625E-07
−21	4,76837158203125E-07	1	4,76837158203125E-07
−22	2,38418579101563E−07	0	0
−23	1,19209289550781E−07	0	0
−24	5,96046447753906E−08	1	5,960464477539063E-08
−25	2,98023223876953E−08	1	2,9802322387695313E-08
−26	1,49011611938477E−08	0	0
−27	7,45058059692383E−09	1	7,450580596923828E-09
	sum of products:		**0,100000001490116119384765625000**

Note: calculation done in Gnumeric, which shows all digits of the result; Microsoft Excel and LibreOffice only show the value 0,100000001490116 (but the correct full value is preserved)

2.5 Floating-Point Representation - IEEE-754 Standard

- example of double precision without exact representation
  - |0|01111111011|1001100110011001100110011001100110011001100110011001100110011010| =
    + 1.1001100110011001100110011001100110011001100110011001100110011010$_2$x2^(01111111011$_2$-1023) =
    + 1.1001100110011001100110011001100110011001100110011001100110011010$_2$x2^(1019$_{10}$-1023$_{10}$=-4) =
    + 0.0001100110011001100110011001100110011001100110011001100110011010$_2$x2^0 = … =

    0.1000000000000000055511151231257827021181583404541015625$_{10}$ ≈ 0.1
  - to confirm this value:
    - use https://www.exploringbinary.com/floating-point-converter/
    - print 0.1 as `double` in a C program:

error = 0.00000000000000005551115...

```
#include <stdio.h>
int main() {
  double xx=0.1;
  printf("%.60f\n", xx); // precision of 60 digits with right zeros
  printf("%.60g\n", xx); // precision of 60 digits without right zeros
  printf("%.60g\n", 0.1);// note the implicit conversion of 0.1 to double
}
```

```
0.1000000000000000055511151231257827021181583404541015625000000
0.1000000000000000055511151231257827021181583404541015625
0.1000000000000000055511151231257827021181583404541015625
```

2.5 Floating-Point Representation - IEEE-754 Standard

- **zeros**
  - only zeros in the exponent and mantissa: |0/1| **0 … 0** | **0 … 0** |
  - 2 zeros (+0 and -0); comparing zeros may return False !
  - moreover, in this case the implicit first bit 1 is not used
- **denormalized values**
  - |0/1| **0 … 0** | non-zero | ; without implicit bit
  - to represent numbers that are close to zero
- **special values**
  - only 1s in the exponent: |0/1| **1 … 1** | x … x |
  - mantissa with only zeros: +/- infinite
  - mantissa different from zero: NaN (not-a-number)

2.5 Floating-Point Representation - Representation Errors

- No matter how many bits we use in a floating-point representation, our model must be finite
- The real number system is, of course, infinite, so our models can give nothing more than an approximation of a real value
- At some point, every model breaks down, introducing errors into our calculations
- By using a greater number of bits in our model, we can reduce these errors, but we can never totally eliminate them

2.5 Floating-Point Representation - Representation Errors

- We should try to reduce errors, or at least be aware of their possible magnitude in our calculations
- **Example**:
  - the simple 14-bit model cannot exactly represent the value 128.5, once the significand would require 9 bits: $128.5_{10} = 10000000.\mathbf{1}_2$
  - representing 128.5_{10} in the 14-bit model implies loosing the **low-order bit**, giving a relative error of (128.5-128) / 128.5 = 0.39%
- We must also be aware that errors can compound through repetitive arithmetic operations

2.5 Floating-Point Representation - Representation Errors

- **Example**: in a procedure that repetitively adds 0.5 to 128.5, we would have an error of nearly 2% after only four iterations:
  - [(128.5+0.5+0.5+0.5+0.5) - (128)] / 130.5 = 1.9%
- Floating-point errors can be reduced when we use operands that are similar in magnitude:
  - In the example above, it would have been better to iteratively add 0.5 to itself and then add 128.5 to this sum.
    - 0.5+0.5+0.5+0.5=2; 2 + 128.5 = 130 (looses 0.5, thus the relative error would be (130.5 - 130)/130.5 = 0.38%)
- In these examples, the error relates to the loss of the low-order bit. Loosing the high-order would be more problematic (why?)

2.5 Floating-Point Representation - Representation Errors

- Because of truncated bits, one cannot always assume that a particular floating point operation is commutative or distributive, that is, not always the following holds:

  $(a + b) + c = a + (b + c)$

  $a*(b + c) = ab + ac$

- To test a floating point value for equality to some other number, first figure out how close one number can be to be considered equal. Call this value *epsilon* and use the statement:

  if (abs(x-y)) < epsilon) **then** ...

2.5 Floating-Point Representation - Representation Errors

- The following C programs illustrate some problematic cases

```
#include <stdio.h>
#include <math.h>

int main()
{
 float x=0.1, y=0.2, z=0.3, w=0.4, a, b;

 a=y-x; b=w-z; // a == b == 0.1, right ?
 if (a!=b) {   // unfortunatelly, no !
        printf("%.30g\n", a);
        printf("%.30g\n", b);
 }
 printf("error: %.30g\n", fabs(b-a));
}
```

```
a: 0.100000001490116119384765625
b: 0.099999994039535522460937500
error: 7.450580596923828125e-09
```

```
#include <stdio.h>
#include <math.h>
//#define EPSILON 1.0e-16
#define EPSILON 1.0e-3

int main()
{
 double a=0.1, b=0.2, c=0.3, d, e, error;

 d=(a+b)+c; e=a+(b+c); error=fabs(d-e);
 printf("d:%.60g\n", d); printf("e:%.60g\n", e);
 printf("error:%.60g\n", error);
 if (error < EPSILON) printf("values considered equal\n");
 else printf("values considered different\n");
}
```

```
d:0.600000000000000088817841970012523233890533447265625
e:0.599999999999999977795539507496869191527366638183593750
error:1.110223024625156540423631668090820312500000000e-16
values considered equal
```

2.5 Floating-Point Representation - Representation Errors

- **A real-world case (*)**:
  - the following program was being used by an odometer to increment a counter (meters) per each 0,01m (1 cm) traveled; but after 10 000 Km, the distance registered is much lower !

```
#include <stdio.h>

int main() {
    float meters = 0;
    int iterations = 1000000000; // 10000Km / 0,01m = 100000000
    for (int i = 0; i < iterations; i++) meters += (float)0.01;
    printf("Expected: %.30g meters\n", (float)0.01*iterations);
    printf("Obtained: %.30g meters\n", meters;
}
```

```
Expected: 10000000 meters
Obtained: 262144 meters
```

After reaching 262144, all further increments will have no effect !

(*) https://jvns.ca/blog/2023/01/13/examples-of-floating-point-problems/

2.5 Floating-Point Representation - Representation Errors

- **A real-world case**: **explanation**
  - in IEEE-754 only certain numbers are represented exactly; moreover, as numbers increase, the distance between the numbers that are exactly representable will also increase
  - thus, a number *x* is represented by the closest number *y* that is exactly representable; also, *y* is chosen by rounding up *x*
  - right after *x*=262144.0 the next representable number is *y*=262144.03125 [and the next ones are at the same distance, equal to 0.03125, until reaching 524287.96875; then, the distance between representable numbers doubles (see (*))
  - now, *x* + 0.01 = 262144.01; but for this number to be converted to *y*=262144.03125, instead of *x*+0.01 we should have at least *x* + (0.03125/2) = *x* + 0.015625 ...

(*) https://float.exposed/0x48800000

2.5 Floating-Point Representation - Representation Errors

checkpoint: exercise 2*.8

- **A real-world case: solutions**

```
#include <stdio.h>
#include <math.h>

int main() {
    double meters = 0;
    int iterations = 1000000000;
    for (int i = 0; i < iterations; i++) meters += (double)0.01;
    printf("Expected: %.30g meters\n", (double)0.01*iterations);
    printf("Obtained: %.30g meters\n", meters;
    printf("Error: %.30g meters\n", fabs((double)0.01*iterations-meters));
}
```

```
Expected: 10000000 meters
Obtained: 9999999.8251586686819791793823 2 meters
Error: 0.17484133131802082061767578125 meters
```

Not yet equal, but a much smaller error.

```
#include <stdio.h>
#include <math.h>

int main() {
    double meters = 0;
    int iterations = 1000000000 / 50;
    for (int i = 0; i < iterations; i++) meters += (double)0.5;
    printf("Expected: %.30g meters\n", (double)0.5*iterations);
    printf("Obtained: %.30g meters\n", meters);
    printf("Error: %.30g meters\n", fabs((double)0.5*iterations-meters));
}
```

```
Expected: 10000000 meters
Obtained: 10000000 meters
Error: 0 meters
```

By changing the increment, the error was completely eliminated.

2.5 Floating-Point Representation - Representation Errors

- Floating-point *overflow* and *underflow* are other problems, that may lead programs to crash
- *Overflow* occurs when there is no room to store the high-order bits resulting from a calculation
- Underflow occurs when a value is too small to store, possibly resulting in division by zero

Is it better that a program silently produces wrong results, or that suddenly crashes ?

2.6 Character Codes

- Calculations aren't useful until their results can be displayed in a manner that is meaningful to people
- We also need to store the results of calculations, and provide a means for data input
- Thus, human-understandable characters must be converted to computer-understandable bit patterns using some sort of character encoding scheme
- As computers have evolved, character codes have evolved. Larger computer memories and storage devices permit richer character codes

2.6 Character Codes - BCD and EBCDIC

- The earliest computer coding systems used six bits
- Binary-coded decimal (BCD) was one of these early codes. It was used by IBM mainframes in the 50s/60s
- In 1964, BCD was extended to an 8-bit code, EBCDIC (Extended Binary-Coded Decimal Interchange Code)
- EBCDIC was one of the first widely-used computer codes that supported upper *and* lowercase alphabetic characters, in addition to special characters, such as punctuation and control characters
- EBCDIC and BCD are still in use by IBM mainframes

2.6 Character Codes - EBCDIC

	Digit															
Zone	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	NUL	SOH	STX	ETX	PF	HT	LC	DEL		RLF	SMM	VT	FF	CR	SR	SI
0001	DLE	DC1	DC2	TM	RES	NL	BS	IL	CAN	EM	CC	CU1	IFS	IGS	IRS	IUS
0010	DS	SOS	FS		BYP	LF	ETB	ESC			SM	CU2		ENQ	ACK	BEL
0011			SYN		PN	RS	UC	EOT				CU3	DC4	NAK		SUB
0100	SP										[	.	<	(	+	!
0101	&										]	$	*	)	;	^
0110	–	/									¦	,	%	_	>	?
0111										'	:	#	@	'	=	"
1000		a	b	c	d	e	f	g	h	i						
1001		j	k	l	m	n	o	p	q	r						
1010		~	s	t	u	v	w	x	y	z						
1011																
1100	{	A	B	C	D	E	F	G	H	I						
1101	}	J	K	L	M	N	O	P	Q	R						
1110	\		S	T	U	V	W	X	Y	Z						
1111	0	1	2	3	4	5	6	7	8	9						

- Easy conversion lowercase ↔ uppercase:
  - just negate the 2nd most significant bit ...

2.6 Character Codes - EBCDIC

NUL	Null	TM	Tape mark	ETB	End of transmission block
SOH	Start of heading	RES	Restore	ESC	Escape
STX	Start of text	NL	New line	SM	Set mode
ETX	End of text	BS	Backspace	CU2	Customer use 2
PF	Punch off	IL	Idle	ENQ	Enquiry
HT	Horizontal tab	CAN	Cancel	ACK	Acknowledge
LC	Lowercase	EM	End of medium	BEL	Ring the bell (beep)
DEL	Delete	CC	Cursor Control	SYN	Synchronous idle
RLF	Reverse linefeed	CU1	Customer use 1	PN	Punch on
SMM	Start manual message	IFS	Interchange file separator	RS	Record separator
VT	Vertical tab	IGS	Interchange group separator	UC	Uppercase
FF	Form Feed	IRS	Interchange record separator	EOT	End of transmission
CR	Carriage return	IUS	Interchange unit separator	CU3	Customer use 3
SO	Shift out	DS	Digit select	DC4	Device control 4
SI	Shift in	SOS	Start of significance	NAK	Negative acknowledgement
DLE	Data link escape	FS	Field separator	SUB	Substitute
DC1	Device control 1	BYP	Bypass	SP	Space
DC2	Device control 2	LF	Line feed		

- In the special characters, you can recognize some still used today(NUL, DEL, CR, NL, BS, ESC, SP) and others already obsolete (PF e PN, for punched cards)

2.6 Character Codes - ASCII

- Other manufacturers, more concerned with data transmission standards between systems, have adopted the **ASCII** (*American Standard Code for Information Interchange*), with 7 bits, in substitution of the proprietary codes of 6 and 8 bits
- ASCII is derived from telecommunication (telex) codes
- Until recently, ASCII code was the dominant code outside the world of IBM mainframes

2.6 Character Codes - ASCII

0	NUL	16	DLE	32		48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	TAB	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	l
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

NUL	Null	DLE	Data link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell (beep)	ETB	End of transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed, new line	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed, new page	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
		DEL	Delete/Idle

- 32 control characters(NUL, SOH, ...), 10 digits (0-9), 52 letters (a-zA-Z), 32 special characters ($, #, ...), space
- 8º bit used as a parity bit, for error detection purpose
  - Less usefulness <, because of > more computer reliability
  - 8º bit bit is used to encode more special characters(ç,ã,...)

2.6 Character Codes
- Parity bit

- The most basic form of error detection
- **Even parity bit**: 1 if ("number of 1s" + 1) is even
- **Odd parity bit**: 1 if ("number of 1s" + 1) is odd

7 bits of data (number of 1s)	8 bits (including **parity bit**)	
	EVEN parity	**ODD parity**
0000000 (0)	**0** 0000000	**1** 0000000
1010001 (3)	**1** 1010001	**0** 1010001
1101001 (4)	**0** 1101001	**1** 1101001
1111111 (7)	**1** 1111111	**0** 1111111

2.6 Character Codes - Unicode

- EBCDIC and ASCII are based on the Latin alphabet, used by a minority of the world's population
- Many of today's systems opted for Unicode, a 16-bit system that allows the encoding of the characters of any language
  - the Java language and some operating systems use Unicode as their default character code
  - the Unicode coding space is divided into 6 parts; the first part is for Western alphabets, including Latin, Greek, and Cyrillic

2.6 Character codes - Unicode

- The Unicode codes-pace allocation is shown at the right →
  - the lowest numbered characters include the ASCII code
  - the higher numbered characters allow anyone to define its own codes

Character Types	Language	Number of Characters	Hexadecimal Values
Alphabets	Latin, Greek, Cyrillic, etc.	8192	0000 to 1FFF
Symbols	Dingbats, Mathematical, etc.	4096	2000 to 2FFF
CJK	Chinese, Japanese, and Korean phonetic symbols and punctuation.	4096	3000 to 3FFF
Han	Unified Chinese, Japanese, and Korean	40,960	4000 to DFFF
	Han Expansion	4096	E000 to EFFF
User Defined		4095	F000 to FFFE

checkpoint:

exercises 2.24 to 2.26

Chapter 2 Conclusion

- Computers store data in the form of bits, bytes, and words using the binary numbering system
- Hexadecimal numbers are formed using four-bit groups called nibbles (or nybbles)
- Signed integers can be stored in one's complement, two's complement, or signed magnitude representation
- Floating-point numbers are usually coded using the IEEE 754 floating-point standard
- Floating-point operations are not necessarily commutative or distributive
- Character data is stored using ASCII, EBCDIC, or Unicode

References

- book ECOA – Chapter 2
- http://en.wikipedia.org/wiki/Binary_numeral_system
- http://en.wikipedia.org/wiki/Octal
- http://en.wikipedia.org/wiki/Hexadecimal
- http://en.wikipedia.org/wiki/Signed_number_representations
- http://en.wikipedia.org/wiki/One%27s_complement
- http://en.wikipedia.org/wiki/Two%27s_complement
- http://en.wikipedia.org/wiki/Floating-point_arithmetic
- http://en.wikipedia.org/wiki/IEEE_754-2008
- http://en.wikipedia.org/wiki/Unicode
- http://en.wikipedia.org/wiki/Ascii
- http://en.wikipedia.org/wiki/Binary-coded_decimal
- http://en.wikipedia.org/wiki/Ebcdic